

Lecture 10: The Producer Consumer Problem

Vivek Kumar

Computer Science and Engineering

IIIT Delhi

vivekk@iiitd.ac.in

Last Lecture

```

int main() {
    int fd[2], status;
    pipe(fd);
    if(fork() == 0) {
        /* Child process */
        close(fd[0]);
        char buff[] = "Hello my dear good Parent";

        write(fd[1], buff, sizeof(buff));
        exit(0);
    }
    /* Parent process */
    close(fd[1]);
    char buff[100];
    read(fd[0], buff, sizeof(buff));
    printf("My obedient child says: %s\n", buff);
    wait(NULL);
    return 0;
}

```

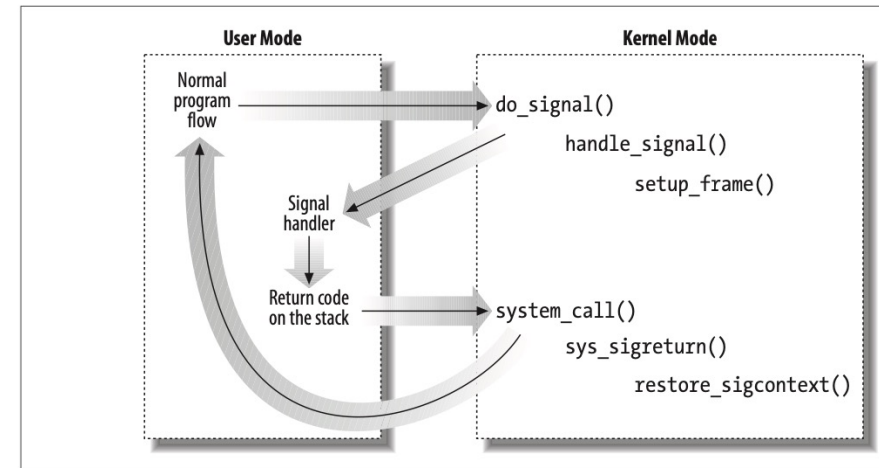
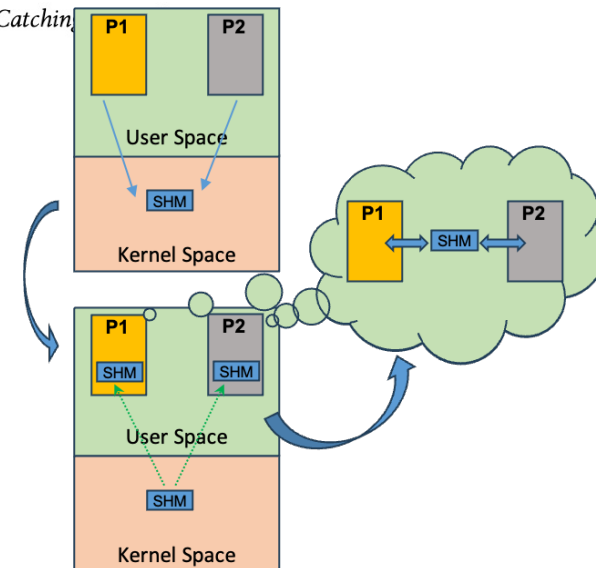


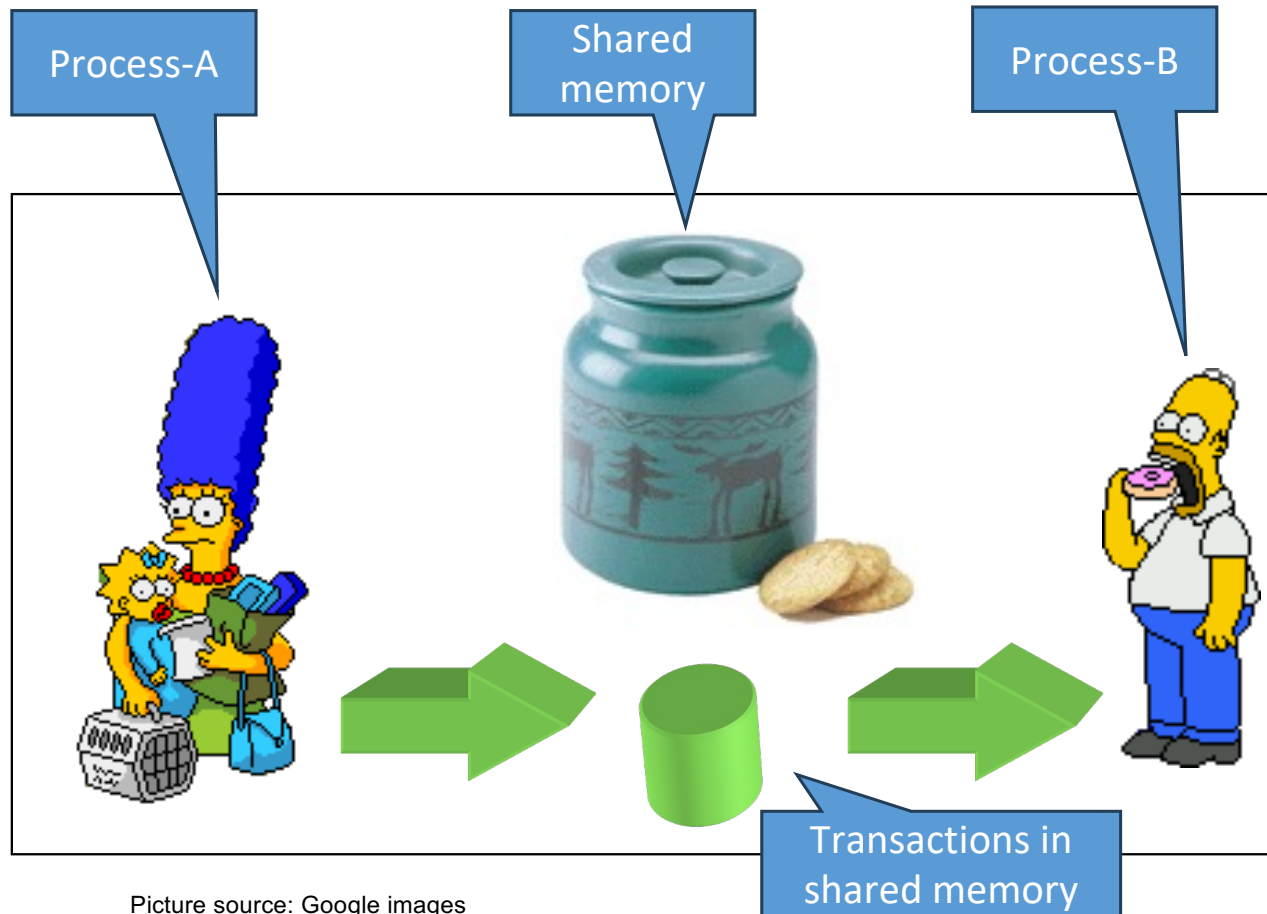
Figure 11-2. Catching



Today's Class

- Producer consumer problem

The Producer Consumer Problem



Picture source: Google images

- The Simpsons story
 - Homer is fond of eating cookies every time
 - Marge is super busy with her baby and so cannot bake a lot of cookies at once
 - The cookie jar is the place to store cookies shared between Marge and Homer
 - Homer picks a cookies from cookie jar if available and then waits for Marge to prepare one for him
 - Marge waits for Homer to eat the cookie from cookie jar and prepares one if no cookie is available
- We need to synchronize between transactions, for example, the producer-consumer scenario described above (or bounded buffer problem)

The Producer Consumer Problem

```
[vivek@possum:~/os23$ ./a.out
Marge bake Cookie-1
Homer ate Cookie-1
Marge bake Cookie-2
Homer ate Cookie-2
Marge bake Cookie-3
Homer ate Cookie-3
Marge bake Cookie-4
Homer ate Cookie-4
Marge bake Cookie-5
Homer ate Cookie-5
```

- The Simpsons story
 - Homer is fond of eating cookies every time
 - Marge is super busy with her baby and so cannot bake a lot of cookies once
 - The cookie jar is the place to store cookies shared between Marge and Homer
 - Homer picks a cookies from cookie jar if available and then waits for Marge to prepare one for him
 - Marge waits for Homer to eat the cookie from cookie jar and prepares one if no cookie is available
- We need to synchronize between transactions, for example, the producer-consumer scenario described above (or bounded buffer problem)

The Producer Consumer Problem

```
typedef struct cookiejar_t {
    int cookie;
    int empty;
} cookiejar_t;

cookiejar_t* cookiejar;

int main() {
    cookiejar = setup();
    cookiejar->empty=1;
    if(fork() == 0) { homer(); }
    if(fork() == 0) { marge(); }
    wait(NULL); // wait for Homer process
    wait(NULL); // wait for Marge process
    cleanup();
    return 0;
}
```

1) shm_open (...)
2) ftruncate (...)
3) mmap (...)

1) munmap (...)
2) close (...)
3) shm_unlink (...)

- We will create a shared memory segment of the type `cookiejar_t` that contains the “cookie” number and a flag “empty” to indicate the availability status of a cookie in the cookie jar
 - 0 = available
 - 1 = unavailable
- Setup and cleanup of shared memory region works exactly as described in earlier slides

The Producer Consumer Problem

```
void homer() {
    for(int i=0; i<5; i++) {
        while(cookiejar->empty) {
            /*Loop endlessly*/
        }
        printf("Homer ate Cookie-%d\n", cookiejar->cookie);
        cookiejar->empty = 1;
    }
    cleanup_and_exit();
}
```

```
void marge() {
    for(int i=0; i<5; i++) {
        while(!cookiejar->empty) {
            /*Loop endlessly*/
        }
        printf("Marge bake Cookie-%d\n", ++cookiejar->cookie);
        cookiejar->empty = 0;
    }
    cleanup_and_exit();
}
```

```
vivek@possum:~/os23$ ./a.out
Marge bake Cookie-1
^C
vivek@possum:~/os23$ ./a.out
Marge bake Cookie-1
^C
vivek@possum:~/os23$ ./a.out
Marge bake Cookie-1
^C
```

- I tried running the program several times but Homer was simply not eating any cookie despite the fact that Marge had in fact baked one for him..
 - What went wrong?

Compiler Ruined the Show!

```
void homer() {
    for(int i=0; i<5; i++) {
        while(true) {
            /*Loop endlessly*/
        }
        printf("Homer ate Cookie-%d\n", cookiejar->cookie);
        cookiejar->empty = 1;
    }
    cleanup_and_exit();
}
```

```
void marge() {
    for(int i=0; i<5; i++) {
        while(false) {
            /*Loop endlessly*/
        }
        printf("Marge bake Cookie-%d\n", ++cookiejar->cookie);
        cookiejar->empty = 0;
    }
    cleanup_and_exit();
}
```

- Compiler did not know that the value of variable “**empty**” is being changed by another process
- It noticed during compilation that main() first sets empty=1 and then calls the methods homer() and marge() where the variable **empty** was simply being tested inside while condition
- It acts oversmart and carries out optimization by simply replacing the variable “**empty**” with 1 at all places
- **Fix?**
 - Declare the variables “empty” as **volatile**

The Producer Consumer Problem

```
void homer() {
    for(int i=0; i<5; i++) {
        while(cookiejar->empty) {
            /*Loop endlessly*/
        }
        printf("Homer ate Cookie-%d\n", cookiejar->cookie);
        cookiejar->empty = 1;
    }
    cleanup_and_exit();
}
```

```
void marge() {
    for(int i=0; i<5; i++) {
        while(!cookiejar->empty) {
            /*Loop endlessly*/
        }
        printf("Marge bake Cookie-%d\n", ++cookiejar->cookie);
        cookiejar->empty = 0;
    }
    cleanup_and_exit();
}
```

```
vivek@possum:~/os23$ ./a.out
Marge bake Cookie-1
Homer ate Cookie-1
Marge bake Cookie-2
Homer ate Cookie-2
Marge bake Cookie-3
Homer ate Cookie-3
Marge bake Cookie-4
Homer ate Cookie-4
Marge bake Cookie-5
Homer ate Cookie-5
```

- **volatile** seems to have fixed the problem?
 - It is an incorrect solution
 - What happens when more than one processes are racing for updating the variable "empty"?
 - **Race condition!**

Semaphores to the Rescue!

```
typedef struct cookiejar_t {
    int cookie;
    sem_t jar_empty;
    sem_t jar_full;
} cookiejar_t;

cookiejar_t* cookiejar;

int main() {
    cookiejar = setup();
    cookiejar->empty=1;
    sem_init(&cookiejar->jar_empty, 1, 1);
    sem_init(&cookiejar->jar_full, 1, 0);
    if(fork() == 0) { homer(); }
    if(fork() == 0) { marge(); }
    wait(NULL); // wait for Homer process
    wait(NULL); // wait for Marge process
    sem_destroy(&cookiejar->jar_empty);
    sem_destroy(&cookiejar->jar_full);
    cleanup();
    return 0;
}
```

- We will declare two semaphore variables inside the shared memory region so as they can be shared by both Marge and Homer processes
- Main process will call the **init** and **destroy** APIs on these semaphore objects

Semaphores to the Rescue!

```
void homer() {
    for(int i=0; i<5; i++) {
        sem_wait(&cookiejar->jar_full);
        printf("Homer ate Cookie-%d\n", cookiejar->cookie);
        sem_post(&cookiejar->jar_empty);
    }
    cleanup_and_exit();
}
```

```
void marge() {
    for(int i=0; i<5; i++) {
        sem_wait(&cookiejar->jar_empty);
        printf("Marge bake Cookie-%d\n", ++cookiejar->cookie);
        sem_post(&cookiejar->jar_full);
    }
    cleanup_and_exit();
}
```

```
vivek@possum:~/os23$ ./a.out
Marge bake Cookie-1
Homer ate Cookie-1
Marge bake Cookie-2
Homer ate Cookie-2
Marge bake Cookie-3
Homer ate Cookie-3
Marge bake Cookie-4
Homer ate Cookie-4
Marge bake Cookie-5
Homer ate Cookie-5
```

- At the start, the value of semaphores **jar_full** and **jar_empty** was set to 0 and 1, respectively
- Homer process upon creation will block in `sem_wait` as the value of `jar_full` was initially 0 (decremented to -1)
- Marge upon activation will decrement `jar_empty` to 0 and will not block. It will bake a cookie, increment the `jar_full` (now its zero), and wake up Homer from `sem_wait`. Finally, it will block inside `sem_wait` after decrementing `jar_empty` (now its -1)
- Homer will awake from `sem_wait`, eat the cookie, increment the `jar_empty` semaphore (now its 0), and wake up Marge from `sem_wait`. Finally, it will block inside `sem_wait` after decrementing `jar_full` (now its -1)
- And the cookie business continues for five times..
- We will revisit the topic of mutual exclusion during lectures on concurrency (multithreading), where we will again discuss it in depth!

Next Lecture

- Introduction to Process Scheduling
- Regular quiz-2 on Wednesday (16th Sep)
 - Syllabus: Lectures 05-10